T20 Token Optimization

Technical Intermediate

Reducing token consumption without losing quality

RTK: Upstream filtering

RTK (Rust Token Killer) intercepts command outputs **before** they enter Claude's context. It is not a post-hoc summary, it is a filter that eliminates structurally redundant noise.

```
# Installation
brew install rtk
# or: cargo install --git https://github.com/rtk-ai/rtk

# Inspect configuration changes before installing the integration
rtk init --dry-run
rtk init
```

Measure output reduction

Compare equivalent commands on the same repository state, inspect failures, and use `rtk gain` for local estimates. Smaller shell output is only one contributor to token usage; it does not establish an equal reduction in the total bill. RTK estimates token counts rather than running the model's tokenizer. See the RTK savings explanation.

Essential RTK commands

```
rtk git status      # Condensed git
rtk git log         # Clean log output
rtk cargo test      # Filtered Rust tests
rtk vitest run      # Condensed JS tests
rtk gh pr view 123  # Optimized GitHub PR
rtk gain            # Savings dashboard
```

Compact: Freeing up context

`/compact` summarizes conversation history. The recovered space varies and details can be lost; preserve decisions and next steps in files and inspect `/context` before continuing.

```
/compact            # Summarize the current session
```

The difference from `/clear`: compact preserves session continuity, clear starts from scratch.

Limit reads and search scope

Claude Code does not document a native `.claudeignore` mechanism. Name the relevant directories in requests. Where access must be blocked, use permission rules in `.claude/settings.json`, for example:

```
{
  "permissions": {
    "deny": ["Read(./node_modules/**)", "Read(./dist/**)"]
  }
}
```

Review shell and MCP access separately. These rules are access restrictions, not a ranking hint for search. See permission patterns.

Efficient prompt rules

Target, don't load:

- "Check the `login` function in `auth.ts:45-60`" rather than "Check auth.ts for issues"
- Reference symbols (`calculateTotal`) rather than entire files
- Avoid pasting large JSON blobs or raw logs into the prompt

What consumes the most:

Action	What to measure
Reading a large file	Returned text size
MCP call	Tool schemas and returned result
Long conversation	History retained in context
Test run	Output returned, including failures

Combined strategy

Filter verbose shell output, compact when useful, and restrict reads to relevant sources. Verify the resulting output still contains the evidence needed to complete the task.